Deciding How Many Floor Types a Tower Needs: An Exact Feasibility Method for Residential Unit Programs

Yiliang Shao*

Abstract

Residential towers are delivered through repeated floor products: each additional floor template carries coordination, documentation, and procurement consequences, so a unit program must fit a limited, sufficiently repeated template kit. This paper decides exactly whether it does, separating optimal, infeasible, and not-solved outcomes and, for an infeasible program, isolating the requirement that binds. The model carries a building-wide template cap, a minimum production run per template, heterogeneous plates, and continuous unit-area bands, made tractable at whole-tower scale by a count-vector reformulation. A synthetic stress test proves three templates impossible and four sufficient, with template variety rather than plate tolerance binding; a bounded-count relaxation admits 9.3 to 13.5 percent more units. On De Piek, a documented Rotterdam tower, the method certifies a compact stacking at fully locked plates. The result is an exact instrument for testing whether a program is deliverable as a repeatable floor-product system.

Keywords: residential development; unit-mix stacking; floor-template repetition; infeasibility diagnosis; exact optimization; constructability; early-stage design

1 Introduction

1.1 Tower unit-mix stacking

Residential towers are delivered through repeated floor products. A developer may test many distributions of studios, one-, two-, and larger family units, but each additional floor template is a product variant that carries coordination, procurement, and construction consequences. The economics are documented. A United Nations survey of repetition in site operations recorded the productivity gain across postwar

*Skidmore, Owings & Merrill, 224 South Michigan Avenue, Suite 1000, Chicago, IL 60604, USA. Email: yiliang.shao@som.com

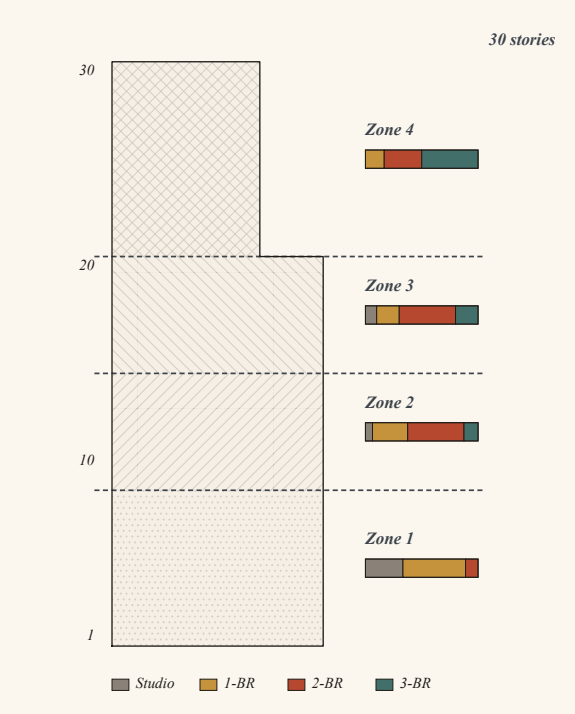

Figure 1: Development-rationalized unit-mix stacking: assemble F floors from at most K repeatable floor products, require each selected product to serve a minimum number of floors, and respect the program and plate-area budgets.

European housing programmes [1], and learning-curve models of construction productivity later gave that gain a quantitative form [2]. Industrialized construction rests its cost case on the same premise of repeatable products [3]. The unit-mix decision must therefore settle whether the program can be carried by a limited and sufficiently repeated template kit.

Unit-mix stacking is the problem of selecting those repeatable floor products, giving each a sufficient production run of floors, and meeting the building-wide program and plate-area budgets (Figure 1). A practical stack must satisfy program-mix targets, planning constraints on floor area, inclusionary-housing minimums, market caps on dominant unit types, and a building-wide limit on product variety. It must also distinguish a structurally infeasible brief from a search that merely ran out of time. This paper develops an exact treatment that operates at the whole-tower scale, admits heterogeneous floor plates, treats unit areas as the *bands* architects specify, limits both the number of templates and the number of floors each must serve, and reports infeasibility honestly.

1.2 Why the problem is computationally hard

Two paths present themselves. *Heuristic search* (genetic algorithms, simulated annealing, generative-ML pipelines) scales to large instances and produces plausible designs. What it cannot do is separate a structurally infeasible specification from a search that merely failed, and that is the distinction an architect needs when negotiating a program against constraints. The experiments make the cost concrete: a genetic algorithm returns a comparable handful of constraint violations whether the program is provably infeasible or merely hard to search (the "Metaheuristic baseline" section). *Exact optimization* through integer programming preserves that distinction.

This paper pursues the exact route. The difficulty sits in the formulation: written the natural way, the feasibility model is intractable at whole-tower scale. A reformulation, developed in the methodology, restores tractability and lets the same exact verdict be computed fast enough, on realistic programs, to sit inside a design iteration.

1.3 What is missing in current approaches

The unresolved development question is whether a unit program fits the delivery logic of repeated floor products. A unit program may fit the available floor areas and still require too many distinct templates, or spread them so thinly that no template serves enough floors to justify its coordination burden. The relevant feasible set is therefore bounded by product variety and repetition as well as area and unit counts.

Current tools generate candidates rather than verdicts. Delve searches massing and program options against performance targets [4], Forma supports early-stage massing studies [5], Finch derives layouts from plate geometry [6], and Fan et al. allocate modular units heuristically [7]. None characterizes the development-feasibility envelope. The missing capability is an exact verdict on whether the program can be delivered as a compact and repeatable stacking system, plus ranked, design-actionable repair guidance when it cannot.

The significance is for the design process, not for the solver. A team that today infers feasibility from whichever candidates a heuristic surfaces instead carries a verdict into the room where the brief is negotiated and, when the answer is no, a ranked relaxation naming the smallest move that makes the brief possible (Figure 2).

1.4 Contributions

The contribution to design decision-making is a development-specific definition of feasibility. A deliverable unit mix fits the plate and program constraints through a limited kit of repeated floor products. Spatial layout follows as a downstream test-fit step (the "Limitations" section).

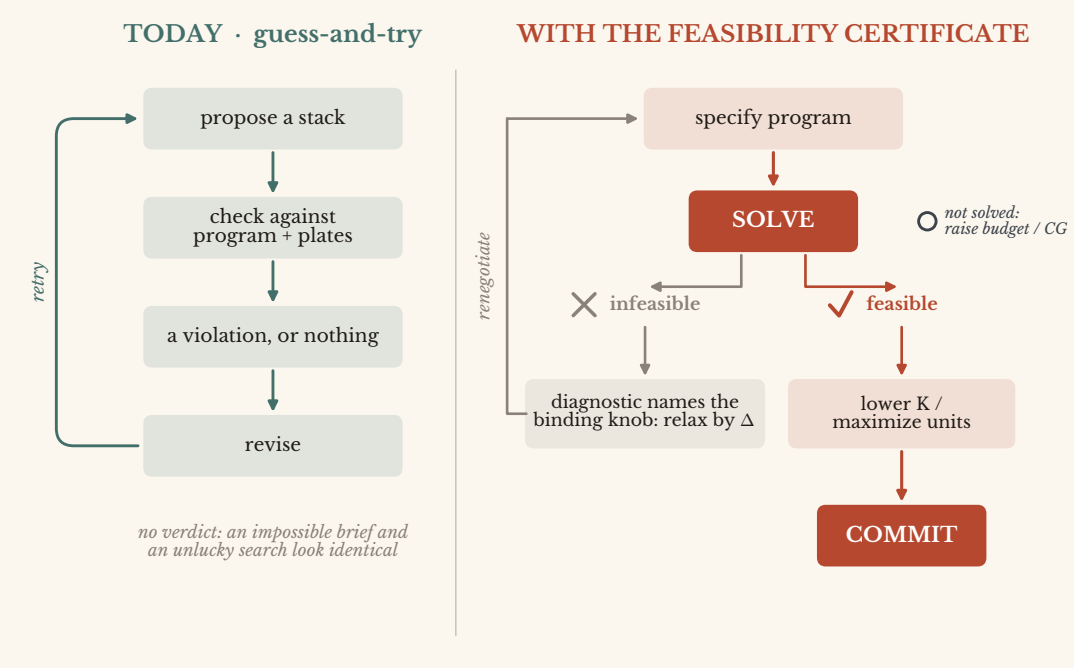

Figure 2: From guess-and-try to a feasibility decision. Today (left), the designer proposes, checks, and revises in a loop that carries no verdict: an impossible brief and a timed-out search look identical. With the instrument (right), the program returns a three-status verdict (*optimal*, *infeasible*, *not solved*). A feasible program flows to a committed decision; an *infeasible* one routes through the single-knob diagnostic, which rules out the requirements that are not binding and so narrows where relaxation must come from.

Four contributions deliver this capability:

1. *Development-rationalized stacking*: a building-wide cap on distinct templates, supported by a minimum number of floors per template, makes product variety part of the feasibility problem.

2. *Feasibility verdict and infeasibility diagnostic*: a three-status verdict (*optimal*, *infeasible*, *not solved*) separates structural infeasibility from search failure, while a single-knob diagnostic converts an *infeasible* verdict into design guidance, ruling out the knobs that cannot restore feasibility and isolating those that can (the "Objectives" section, the "Infeasibility diagnostic" section).

3. *Method*: a count-vector reformulation makes exact treatment tractable when repeated-floor multiplicities and continuous unit-area bands would otherwise create bilinear couplings (the "Methodology" section).

4. *Validation*: on a synthetic 55-floor tower the diagnostic rules out area tolerance and pins template variety as the binding knob, a verdict a hand-derived Diophantine argument confirms; on the public De Piek envelope the same machinery certifies a

compact stacking at fully locked plates (the “Computational Experiments” section, the “Real-building study” section).

2 Background and Related Work

Tower unit-mix stacking sits at the meeting point of literatures that have not previously met on this problem: configuration-IP and variable-sized bin packing in operations research, computational layout design in AEC, and the decision-support tradition in design research.

2.1 Unit-mix stacking in practice

In practice the problem is solved by spreadsheet iteration and, increasingly, by commercial generative-design tools. Sidewalk Labs’ Delve [4], Autodesk Forma (formerly Spacemaker) [5], Finch3D [6], and Architechtures [8] generate and rank candidate floor plans and stacks through proprietary search and scoring pipelines whose internals are not published.

The closest published exact comparator is Aristi Reina et al. [9]. Their integer program assigns fixed-area unit types to the individual floors of three geometrically differentiated high-rise volumes under area, facade-length, mix, and altitude-dependent restrictions. It demonstrates that exact floor-by-floor unit-mix allocation is practical and connects the result to a Grasshopper workflow. Repetition enters that model as given geometry rather than as a decision: the scheme is pre-divided into three standard sections recurring a fixed number of times across the towers, and within a section every level is an independent decision row. No variable activates a floor product, so nothing constrains how many distinct products a solution uses or how many floors each must serve.

The broader academic line remains predominantly heuristic or learned: a two-stage genetic algorithm for modular high-rise residential buildings [7], GA/deep-Q-network aggregation [10], Monte Carlo tree search [11, 12], agent-based modeling with a conditional GAN [13], and heuristic within-unit layout [14]. The surrounding generative-ML tradition, surveyed by Weber et al. [15], ranges from graph-conditioned floorplan learning [16] to graph-constrained GAN layout synthesis [17]. Bilge and Yaman optimize building form together with the unit mix through NSGA-II, reporting limited mix diversity [18]. Practitioner work comes closest to the repetition objective: Botham [19] targets client-specified percentages and folds the counts into the fewest unique floor plans, but publishes neither a method nor a feasibility verdict.

These methods establish the value of unit-mix generation. This paper supplies the information they leave outside the model: whether a program remains feasible under

a cap on floor-product variety, and what must change when it does not.

2.2 Decision support and set-based design

As a design instrument rather than a solver, the contribution belongs to the decision-support tradition in design research. Set-based design carries forward sets of feasible alternatives and defers commitment until the trade-offs are understood, rather than converging early on a single point design [20]. The feasibility envelope returned here is such a set: it reports where a program stays buildable, not a single stack. In that it answers the call for computational tools that help designers map and navigate a design space instead of only delivering an answer [21].

2.3 Configuration IP and bin packing

A *configuration IP* indexes variables by pre-enumerated count patterns rather than item-to-bin placements; the lineage runs from Gilmore and Gomory's cutting-stock formulation [22, 23] through the arc-flow reformulation of Valerio de Carvalho and its later refinements [24, 25] and constant-type bin packing [26]. Three later strands matter here. Variable-sized bin packing [27] and its variable-cost extension [28] admit bins of differing capacity, and are the heterogeneous-bin antecedent for the "Heterogeneous plate classes" section. Multiple-choice vector packing [29, 30] lets each item be realized as one of several alternatives. Packing with fragmentation [31] and with controllable item sizes [32] relax the assumption that an item's size is given, and are the nearest antecedents for a size that is decided rather than read off the data. Within the configuration-IP setting, however, item sizes stay fixed data or finite lists, and approximating a continuous interval needs $\mathcal{O}(1/\delta)$ configurations per item at resolution δ. Where a size is allowed to move, it moves as a fragmentation of one item or as a reduction priced per item, not as a continuous attribute shared by every item of a type and held inside a band. The formulation here keeps one continuous area variable per (template, type) under a linear band, over plates of differing capacity, with a cap on how many distinct templates a solution may use. That combination is missing: it is bilinear unless count vectors are enumerated first, and the "Methodology" section supplies it.

2.4 AEC computational design and IP-based approaches

Exact IP has a small but established foothold in AEC computational design. Aristi Reina et al. [9] directly formulate high-rise unit-mix allocation with fixed unit areas and floor-specific capacities. Adjacent applications include urban block massing (Hua

et al. [33]), room-to-floor assignment (Klawitter et al. [34]), single-floor layout (Keatruangkamala and Sinapiromsaran [35]), and facility layout with continuous coordinates (Huchette et al. [36]). Cubukcuoglu et al. [37] optimize hospitals through stacking, zoning, and routing, but solve the stacking step by spectral clustering rather than exact optimization.

Together, this literature establishes exact allocation and optimization across several architectural scales.

2.5 Patent landscape

A US/EU/WIPO patent search surfaced no exact IP-based tower-scale unit-mix stacking method; the closest, US 11,727,173 B2 [38], is a per-floor delta-greedy heuristic on a single plate. The supplemental material carries the full search summary.

2.6 The gap

The missing object is a whole-tower development-feasibility model in which continuous unit-area bands, repeated-floor multiplicities, and a building-wide cap on product variants are solved together. Count-vector enumeration dissolves the resulting count $\times$ area and count $\times$ repetition couplings, making it possible to certify whether the program fits as a compact production system.

3 Problem Formulation

This section states the stacking problem in architectural terms.

3.1 Building, floors, plate classes

A residential tower is a vertical sequence of F floors, each with a net usable area set by the structural system and core. Towers step back as they rise for planning, wind, or skyline reasons. A contiguous block of floors of equal net area is here called a *plate class*; a tower with no setbacks is a single class of F floors.

Each floor hosts an integer count of units of each *unit type* around a shared core; floors sharing a unit-count vector are operationally identical. The architect keeps the number of distinct floor templates (*zones*) small so that structure and service risers are reused.

3.2 Unit types and the constraint envelope

Every requirement an architect places on a tower's unit mix is a knob, in one of three states, *locked*, *bounded*, or *soft* (Figure 3). The solver reports the envelope under whatever combination the architect imposes; it does not silently relax constraints to produce output.

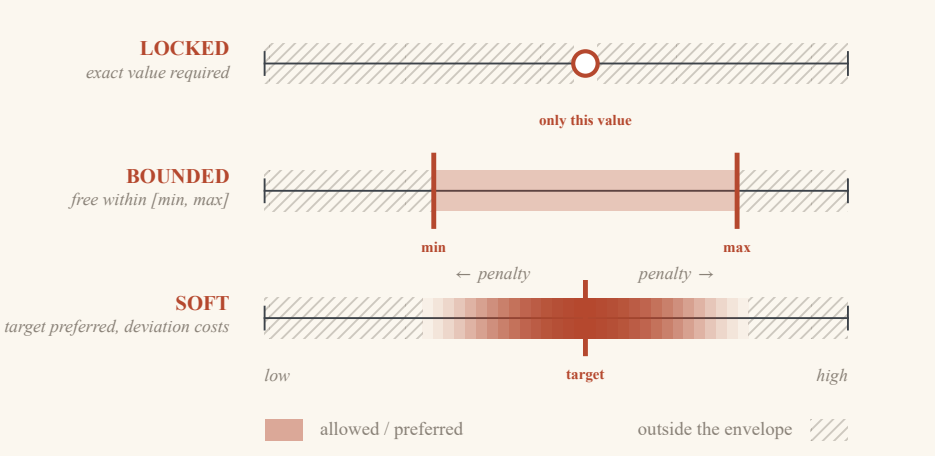

Figure 3: Every requirement is a knob in one of three states: locked (exact value required), bounded (free within $[\min, \max]$), or soft (target preferred, deviation penalized). The constraint envelope is the feasible region under the chosen knob states.

The knobs relevant to unit-mix stacking are:

Floor net area. Each plate class i has target net area A_i with relative tolerance $\tau_i \geq 0$: $\tau_i = 0$ locks the floor area at its structurally determined value, positive τ_i admits a band around the target.

Per-type unit area. Each unit type t has target area $a_t^\star$ with tolerance σ_t. Architects rarely fix unit areas at a point, admitting a band of typically $\pm 5\%$. This continuous within-type attribute creates the bilinearity of the "Direct McCormick MILP (V1)" section.

Building-wide count per type. The total number of units of each type in the building is specified in one of three modes, detailed in the "Three count-specification modes" section.

Number of distinct floor templates K**.** The compactness cap; lower K eases construction and cross-discipline coordination.

Minimum floors per used zone m**.** A one-floor zone is operationally pointless; the architect typically requires at least two floors per used template.

3.3 Three count-specification modes

Three count-specification modes correspond to the three ways architects state programs in practice; all unit types within an instance share one mode.

Proportional. Target shares π_t with $\sum_t \pi_t = 1$, a hard band $\pi_t \pm \beta$ of the total, and a soft L_1 deviation from the ratio: the early-design language ("roughly 20% studios").

Bounded count. Hard per-type floors and ceilings $\underline{N}_t \le N_t \le \overline{N}_t$ with an optional soft pull to a target $N_t^\star$: the negotiated language of binding minimums and caps.

Fixed count. Hard equalities $N_t = N_t^\star$, no soft term: the language of program-locked late development.

Fixed-count is the most demanding; where its envelope closes, the negotiated program is structurally infeasible against the plate areas. Area-mix specifications translate mechanically to count mix via the per-type targets $a_t^\star$; the solver only sees counts.

3.4 Objectives

The objective minimizes deviation from the count specification: in proportional and bounded modes an L_1 penalty against the target ratio or count, weighted so any deviation-free solution dominates a deviation-present one. In fixed-count mode the deviation term vanishes. A secondary *template-simplicity* term, a small positive weight on the number of selected templates, breaks ties toward fewer templates. The objective is otherwise kept minimal; surplus rentable area is not a priority once the per-type bands are met (the "Solver-driven program headroom" section re-introduces a maximize-units objective).

Whatever the mode, a solve returns one of three statuses: *optimal*, a proven optimum; *infeasible*, a proof that no stacking satisfies the specification; or *not solved*, no primal found within the time budget. Experiment tables additionally mark Optimal†: a feasible primal whose dual gap is not closed within budget, a stacking in hand without an optimality certificate.

4 Methodology: The Count-Vector Reformulation

Throughout, T denotes the number of unit types, indexed by $t \in \{1, \dots, T\}$. For a single floor plate of net area A with relative tolerance τ, the floor-area band is $[A_{\text{lo}}, A_{\text{hi}}] = [A(1-\tau), A(1+\tau)]$. Each unit type t has a target area $a_t^\star$ and a per-type tolerance σ_t, giving an area range $[a_t^{\min}, a_t^{\max}] = [a_t^\star(1-\sigma_t), a_t^\star(1+\sigma_t)]$. A tower has F floors organized into at most K zones, sets of floors sharing a common floor template.

4.1 The direct zone-indexed McCormick MILP (V1) and where it fails

The natural formulation is zone-based. Each of the K zones either hosts a block of floors with a common template, or is unused. For zone $z \in \{1, \dots, K\}$ the formulation introduces:

- $f_z \in \mathbb{Z}_+$, the number of floors using zone z (zero if unused);
- $u_z \in \{0, 1\}$, whether zone z is used;
- $n_{t,z} \in \{0, 1, \dots, N_{\max}\}$, the count of unit type t on each floor of zone z;
- $a_{t,z} \in [a_t^{\min}, a_t^{\max}]$, the area assigned to unit type t on each floor of zone z.

The floors must partition into zones,

$$\sum_{z=1}^{K} f_z = F, \tag{1}$$

with the activation link

$$m \cdot u_z \;\le\; f_z \;\le\; F \cdot u_z \qquad \forall z, \tag{2}$$

where m is a minimum-floors-per-used-zone parameter that prevents orphan single-floor zones. The per-floor template area must lie in the plate band,

$$A_{\text{lo}} \cdot u_z \;\le\; \sum_{t=1}^{T} n_{t,z}\, a_{t,z} \;\le\; A_{\text{hi}} \cdot u_z \qquad \forall z. \tag{3}$$

Equation (3) is the source of the trouble. Both $n_{t,z}$ and $a_{t,z}$ are decision variables, so the product $n_{t,z}\, a_{t,z}$ is bilinear. A second bilinearity appears in the building-wide type total: the total of unit type t is

$$N_t \;=\; \sum_{z=1}^{K} n_{t,z}\, f_z, \tag{4}$$

a product of two integer variables.

Both bilinearities admit exact Big-M / McCormick linearization [39]; the full system is in the supplemental material. The variable count scales as $\mathcal{O}(T \cdot K \cdot N_{\max})$, and on representative instances ($T = 4$, $F = 70$, $K = 3$, $N_{\max} = 12$) the model exceeds the $300\,\text{s}$ budget without proving optimality (the "V1 versus V2" section). Tighter linearizations (Glover [40], integer-bilinear cuts, disjunctive variants) operate within the same zone-indexed structure and can lower the constant but not recover the count-vector scaling. This is a structural argument, not an exhaustive empirical claim.

4.2 The count-vector observation

The bilinearity in eq. (3) comes from optimizing *simultaneously* over the integer counts $n_{t,z}$ and the continuous areas $a_{t,z}$. The observation that dissolves it is structural and narrow: once the count vector is fixed, the area-assignment subproblem is a bounded linear program whose feasibility is decided in $\mathcal{O}(T)$.

Proposition 1. *Fix a count vector* $\mathbf{n} = (n_1, \ldots, n_T)$ *and per-type area intervals* $[a_t^{\min}, a_t^{\max}]$. *Areas* $a_t \in [a_t^{\min}, a_t^{\max}]$ *with* $\sum_t n_t\, a_t \in [A_{lo}, A_{hi}]$ *exist if and only if*

$$\sum_t n_t\, a_t^{\min} \le A_{hi} \quad \textit{and} \quad \sum_t n_t\, a_t^{\max} \ge A_{lo}.$$

The test is two dot products, evaluated in $\mathcal{O}(T)$*, and once* $\mathbf{n}$ *is fixed the products* $n_t\, a_t$ *are linear in the area variables.*

Proof. The image of the box $\prod_t [a_t^{\min}, a_t^{\max}]$ under the nonnegative-linear map $\mathbf{a} \mapsto \sum_t n_t\, a_t$ is the interval $[\sum_t n_t\, a_t^{\min}, \sum_t n_t\, a_t^{\max}]$, which meets $[A_{\text{lo}}, A_{\text{hi}}]$ exactly when its lower end does not exceed A_{hi} and its upper end is at least A_{lo}. □

Proposition 1 drives the reformulation. Rather than treating $\mathbf{n}_z$ as decision variables per zone, the method *enumerates* the count vectors that pass the test for a plate, then lets the optimization choose among them (Figure 4); each surviving vector becomes a *template* carrying its own area variables and activation indicator. Enumeration is a bounded depth-first recursion over $t = 1, \ldots, T$ with underflow and overflow pruning against the plate band, both supplied by Proposition 1; A gives the full recursion. Empirically the pool ranges from tens of vectors at $T = 2$ to $> 10^4$ at $T = 6$ (the "Scaling of V2b" section), with enumeration time under $0.15\,\text{s}$ even at the largest pools.

4.3 The single-plate count-vector formulation (V2)

Let $\mathcal{P}$ be the set of feasible count vectors enumerated for a single uniform plate of area band $[A_{\text{lo}}, A_{\text{hi}}]$, with each $\mathbf{n}^p = (n_1^p, \ldots, n_T^p) \in \mathcal{P}$ a distinct *template*. Each template $p \in \mathcal{P}$ carries:

- $x_p \in \mathbb{Z}_+$, the number of floors using template p (zero if not selected);
- $y_p \in \{0, 1\}$, whether template p is selected as a zone;
- $a_{p,t} \in [a_t^{\min}, a_t^{\max}]$, the area assigned to unit type t on template p.

Floors partition into templates,

$$\sum_{p \in \mathcal{P}} x_p \;=\; F. \tag{5}$$

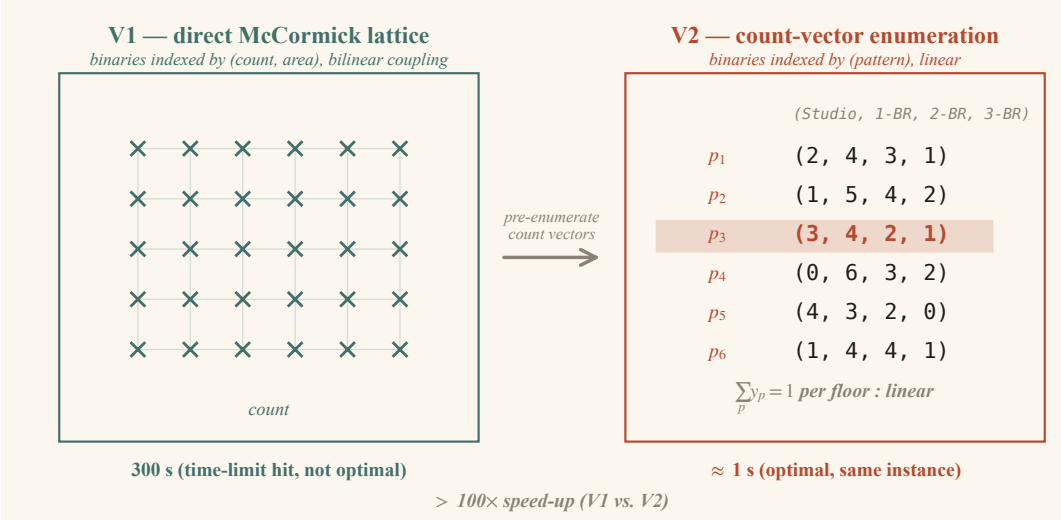

Figure 4: The count-vector reformulation. V1 carries the count-by-area coupling explicitly; V2 pre-enumerates feasible count vectors and selects templates linearly.

A template hosts at most F floors and at least m if selected,

$$m \cdot y_p \;\le\; x_p \;\le\; F \cdot y_p \qquad \forall p, \tag{6}$$

and the number of selected templates is bounded by the compactness cap,

$$\sum_{p \in \mathcal{P}} y_p \;\le\; K. \tag{7}$$

The per-template area band activates only when the template is selected,

$$A_{\text{lo}} \cdot y_p \;\le\; \sum_{t=1}^{T} n_t^p \, a_{p,t}, \tag{8}$$

$$\sum_{t=1}^{T} n_t^p \, a_{p,t} \;\le\; A_{\text{hi}} \cdot y_p \;+\; M_p \,(1 - y_p), \tag{9}$$

where $M_p = \sum_t n_t^p \, a_t^{\max}$ is the template's maximum possible area: a tight, template-specific Big-M computed at enumeration time.

The building-wide total of unit type t is

$$N_t \;=\; \sum_{p \in \mathcal{P}} n_t^p \, x_p, \tag{10}$$

which is linear because n_t^p is a constant. This is the central payoff of the reformulation: the bilinear product in eq. (4) becomes a linear sum of constants times integers, without any McCormick or Big-M product expansion.

The mode-specific count constraints (proportional, bounded, fixed) and the deviation-plus-simplicity objective $\alpha \sum_t (d_t^+ + d_t^-) + \gamma \sum_p y_p$ with $\alpha \gg 1$ are stated formally in the "Three count modes" section. The model is a pure MILP; the only Big-M is the template-tight M_p, and no McCormick product-expansion auxiliaries appear.

The area variables are a feasibility certificate. The continuous areas $a_{p,t}$ enter only the per-template band (8)–(9), absent from the objective and the type totals, and uncoupled across templates. Since every enumerated template already passes Proposition 1, the band is satisfiable for any x_p, y_p and never constrains the integer decisions: the $a_{p,t}$ can be projected out, so the count-vector move absorbs count-by-area feasibility into the enumeration filter rather than optimizing a continuous attribute. Tractability thus comes from the small integer-variable count, not a uniformly tight LP relaxation (the supplemental material treats LP tightness; the observed integrality gap is essentially zero in the one regime where it is meaningful).

4.4 Heterogeneous plate classes

The "Single-plate count-vector formulation (V 2)" section formulated the count-vector model for a single plate class. Real towers are not single-plate. They consist of a main plate stepping back into one or more setbacks, and the K-cap on distinct templates is building-wide.

Let $\mathcal{I}$ index plate classes, each with net area A_i, tolerance τ_i, floor count F_i, and band $[A_i^{\text{lo}}, A_i^{\text{hi}}]$. Count vectors are enumerated per plate by the procedure of the "Count-vector observation" section, giving disjoint pools $\mathcal{P}_i$. Decision variables live over pairs (i, p):

- $x_{i,p} \in \{0, 1, \ldots, F_i\}$, the number of floors of plate i using template p;
- $y_{i,p} \in \{0, 1\}$, whether template p is selected on plate i;
- $a_{i,p,t} \in [a_t^{\min}, a_t^{\max}]$, the area for unit type t on template p of plate i.

Floors partition within each plate,

$$\sum_{p \in \mathcal{P}_i} x_{i,p} \;=\; F_i \qquad \forall i \in \mathcal{I}. \tag{11}$$

Activation and minimum-floors-per-zone act per pair,

$$m \cdot y_{i,p} \;\le\; x_{i,p} \;\le\; F_i \cdot y_{i,p} \qquad \forall i, p. \tag{12}$$

The compactness cap applies across all plates jointly,

$$\sum_{i \in \mathcal{I}} \sum_{p \in \mathcal{P}_i} y_{i,p} \;\le\; K. \tag{13}$$

The per-template area band activates only when the template is selected, and the band parameters are those of plate i:

$$A_i^{\mathrm{lo}}\, y_{i,p} \;\le\; \sum_{t=1}^{T} n_t^{i,p}\, a_{i,p,t} \qquad \forall i, p, \tag{14}$$

$$\sum_{t=1}^{T} n_t^{i,p}\, a_{i,p,t} \;\le\; A_i^{\mathrm{hi}}\, y_{i,p} \;+\; M_{i,p}\,(1 - y_{i,p}) \qquad \forall i, p, \tag{15}$$

where the template-specific Big-M is $M_{i,p} = \sum_t n_t^{i,p}\, a_t^{\max}$.

The building-wide total of unit type t aggregates across all plates,

$$N_t \;=\; \sum_{i \in \mathcal{I}} \sum_{p \in \mathcal{P}_i} n_t^{i,p}\, x_{i,p}, \tag{16}$$

which remains linear because $n_t^{i,p}$ is a constant fixed at enumeration time.

The generalization reintroduces no bilinearity. The type-total aggregator stays linear in the constants $n_t^{i,p}$, and model size grows linearly in $|\mathcal{I}|$; the supplemental material consolidates the formulation.

4.5 The three count modes, formally

The mode semantics of the "Three count-specification modes" section carry over unchanged; the supplemental material gives the formal constraint fragments. In proportional and bounded modes, per-type deviation pairs $d_t^+, d_t^- \ge 0$ measure the distance from the target ratio or count; in fixed-count mode the totals are pinned, $N_t = N_t^\star$. The objective extends to the heterogeneous case with a global selection sum:

$$\min \;\; \alpha \sum_{t=1}^{T} (d_t^+ + d_t^-) \;+\; \gamma \sum_{i \in \mathcal{I}} \sum_{p \in \mathcal{P}_i} y_{i,p}. \tag{17}$$

In fixed-count mode the first term is identically zero and the objective reduces to template simplicity alone; infeasibility then carries unambiguous meaning: the count specification is structurally inconsistent with the plate configuration at the given tolerances.

4.6 Differentiation from prior art

Aristi Reina et al. [9] establish exact floor-by-floor allocation for fixed unit areas; this work extends exact unit-mix optimization to the delivery structure of repeat-floor development: a limited kit of floor products, minimum production runs, continuous unit-area bands, and an explicit infeasibility-and-repair protocol. The pattern-enumeration technique itself is 60 years old [22]; its role here is to make that delivery model tractable within a design iteration. The V1-versus-V2 runtimes (the "V1 versus V2" section) show the computational consequence.

4.7 Composable extensions

Several tower-design constraints compose with the count-vector skeleton without disturbing it. Zone-ordering preferences (larger units higher up) enter as an elevation-weighted objective term that adds no new variables (Figure 7); a hard variant instead adds one integer position variable per zone. Cost or embodied-carbon objectives add per-template constants $c_{i,p}$, and sequencing composes through precedence constraints. Vertical alignment of unit positions does not compose, for reasons taken up in the "Future work" section.

5 The Single-Knob Infeasibility Diagnostic

An exact MILP returns *infeasible* on infeasible inputs, but that status alone leaves the architect with no direction. The single-knob probe works toward the design-actionable statement "relax this one knob by this much and a stacking exists."

Starting from an infeasible instance with parameters $(\boldsymbol{\tau}, \boldsymbol{\sigma}, K, \boldsymbol{N}^{\star})$, the procedure constructs a sequence of perturbed instances each differing from the original in exactly one knob, by a defined step. The probes are:

`plate:i.area_tol`. Widen τ_i by a step $\Delta\tau$ for one plate i, leaving all other plates locked. One probe per plate.

`unit:t.area_tol`. Widen σ_t by a step $\Delta\sigma$ for one unit type t. One probe per type.

`n_zones`. Bump K by a step ΔK.

`bounded:t.window`. In bounded-count mode, widen the $[\underline{N}_t, \overline{N}_t]$ window by a step ΔN for one unit type. One probe per type.

Each probe is solved under a tight time budget ($180\,\mathrm{s}$ here): a probe that returns any feasible solution certifies that a single-knob relaxation of that magnitude restores feasibility, while a probe that remains *infeasible* marks that knob as not binding at that step.

B gives the procedure in full; Figure 5 shows it applied to the case-study $K = 3$ instance (the "K-sweep and the binding knob" section).

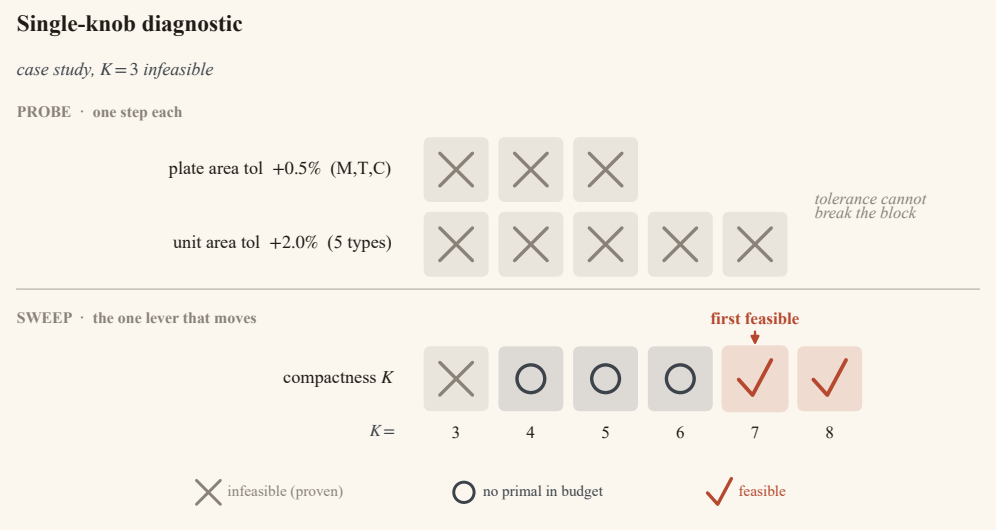

Figure 5: The single-knob diagnostic on the case-study $K = 3$ instance. Every area-tolerance probe remains infeasible, ruling area tolerance out; the $+1$-zone probe returns no primal within its budget, leaving the compactness cap as the open direction. The K-sweep (the "K-sweep and the binding knob" section) completes the verdict: the binding decision is the number of templates, not area tolerance.

Relation to IIS. An Irreducible Infeasible Subsystem [41] returns a mathematically complete minimal constraint set; no wall-clock claim is made against it. The differentiator is granularity: one architect knob maps to many constraints, and IIS returns constraint indices without knob magnitudes. The probe instead answers in the architect's vocabulary ("widen σ_{2B} by 0.02"), at the cost of assuming a single binding knob and stopping short of joint relaxations or optima.

6 Computational Experiments

All headline experiments use the open-source PuLP modelling library [42] with the HiGHS mixed-integer solver [43] on a desktop workstation with no GPU. Where indicated, results are cross-checked on Gurobi 13.0.2 [44] and SCIP [45] under the same modelling layer, separating solver-driven from formulation-driven effects. All reported solve times, statuses, and verdicts are from HiGHS except where attributed to Gurobi or SCIP; the two case-study stacking *arrangements* (Figure 7, Figure 11), which HiGHS does not close cleanly, are computed with Gurobi. Per-cell time budgets are noted per subsection; scripts and per-run environment metadata (solver version, OS, CPU, git commit) live in `experiments/` and `experiments/results/`.

6.1 Direct McCormick (V1) versus count-vector (V2): cost of the natural formulation

This experiment quantifies the runtime cost of choosing the McCormick-style natural formulation (V1) over the count-vector reformulation (V2).

The instances are identical in inputs, hardware, solver, and time budget across the two models; the feasible regions are identical by construction, and the objectives differ only by a V1-only area-encouragement term (see the supplemental material). The headline battery sweeps unit types $T \in \{2, 3, 4, 5\}$ at $F = 70$ floors on a $700\,\mathrm{m}^2$ uniform plate with $\tau = 0.02$ and $K = 3$; the count vector $\boldsymbol{\pi}$ is uniform over T, minimum floors per zone is $m = 2$, and both models cap the per-floor count at twelve. Each cell is replicated across three HiGHS seeds and three values of $\sigma_t \in \{0.02, 0.05, 0.10\}$ (full replication and σ sensitivity in supplemental `v1_vs_v2.csv`). Table 1 and Figure 6 report the headline.

Table 1: V1 versus V2 on identical instances at $F = 70$, $K = 3$, $\sigma_t = 0.05$; times and speedups are medians over three HiGHS seeds, gaps from the instrumented gap probe (seed 42). Optimal† denotes a feasible primal with unclosed dual gap at 300 s.

T	V1 time (s)	V1 status	V1 gap (%)	V2 time (s)	V2 status	$\lvert\mathcal{P}\rvert$	speedup
2	7.0	Optimal	0	0.01	Optimal	13	723×
3	300.1	Optimal†	31.7	0.31	Optimal	132	977×
4	300.1	Optimal†	54.0	1.33	Optimal	453	225×
5	300.1	Optimal†	73.5	58.41	Optimal	835	5×

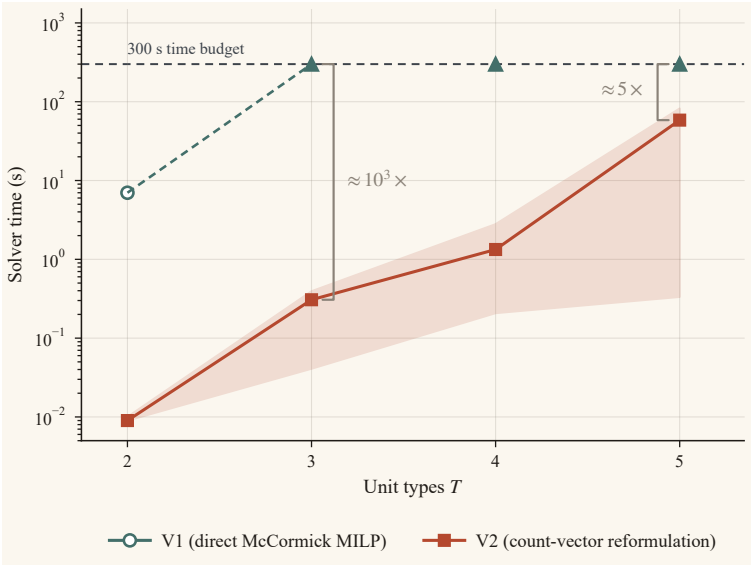

Figure 6: Solve time vs. unit-type count T for V1 and V2 on the same benchmark family. V1 hits the 300 s budget at $T \geq 3$; V2 closes every tested cell.

A sharp formulation-cost cliff sits between $T = 2$ and $T = 3$: V1 closes $T = 2$ in ≈ 7 s

then hits the 300 s budget at every $T \geq 3$ (terminal gaps 31.7–73.5%), while V2 closes every cell to proven optimality in under a minute (Table 1). The median speedups (10^2–10^3 at $T \in \{3, 4\}$, $\approx 5\times$ at $T = 5$) are budget-censored lower bounds, since V1 never proves optimality at $T \geq 3$. Past the tested range the bottleneck shifts to V2's enumeration size, controllable via column generation (the "Scaling of V2b" section, the "Future work" section). A Gurobi cross-check ($T \in \{2, \ldots, 6\}$, 45 cells) confirms the gap is formulation-driven, not solver-specific: Gurobi closes V1 to *optimal* on 44 of 45 cells, yet V2 still beats it by one to three orders of magnitude (supplemental `v1_vs_v2.csv`, `v1_vs_v2_gurobi.csv`).

6.2 Scaling of the multi-plate count-vector model (V2b)

The V2b extension adds plate-class count $|\mathcal{I}|$ and floor distribution as scaling axes. Enumeration stays negligible (≈ 0.11 s even at ~ 22k vectors), but MILP solve time grows sharply with pool size: across the sweep ($|\mathcal{I}| \leq 5$, $T \in \{4, 5, 6\}$), $T = 4$ closes within the 120 s per-cell budget, $T = 5$ leaves a quarter of cells without a primal, and $T = 6$ closes none. Thus T is the dominant scaling lever, and $T \geq 5$ marks where column generation becomes the appropriate tool (the "Future work" section); the full sweep is in supplemental `v2b_scaling.csv`.

7 Case Study: A Synthetic 55-Floor Stress Test

This section and the next bracket the instrument's operating range: here a synthetic stress test presses the formulation against an engineered worst case, and the "Real-building study" section then turns the same machinery on a real building under a comfortable program. The instance is a 55-floor tower whose program is tight enough against the plate envelope that the smallest admissible compactness value is Diophantine-infeasible. It is fully reproducible from its published specification (Table 2, C) with no client-confidentiality constraints. Tolerances are mid-density residential: 0.5% plate, $\pm 5\%$ per type.

7.1 The instance

The tower has three plate classes stepping back as it rises: a wide main plate for street frontage and amenity, a transition, and a crown, all at baseline plate tolerance $\tau_i = 0.005$. Five unit types (studio through 3B, $\sigma_t = 0.05$) carry a fixed-count program of 814 units. Table 2 tabulates the full instance; C lists the scripts.

Table 2: The synthetic case-study instance: plate classes (left) and unit program (right), $\sigma_t = 0.05$ for all types. The program sums to $63{,}484\,\mathrm{m}^2$ against a plate envelope of $62{,}940\,\mathrm{m}^2$ ($\sim 0.9\%$ over nominal, absorbed by the $\pm 5\%$ band); the studio count 58 is the Diophantine blocker at $K = 3$. Count-vector enumeration caps per floor: 8 units per type, 18 in total ($12/26$ for the $K = 8$ stacking figure).

plate label	A_i (m^2)	F_i (floors)	τ_i (baseline)
M (main)	1380	37	0.005
T (transition)	700	12	0.005
C (crown)	580	6	0.005
Total	n/a	55	n/a

type	$a_t^\star$ (m^2)	$N_t^\star$
studio	40	58
1B	55	228
2B	78	264
2B+D	92	120
3B	118	144
Total	n/a	814

7.2 K-sweep and the binding knob

The sweep covers $K \in \{3, \ldots, 11\}$ at baseline $\tau_i = 0.005$ under a $300\,\mathrm{s}$-per-cell budget (Table 3); the $\tau_M = 0$ stress case lives in the envelope sweep (the "Two-dimensional envelope" section). A witness transferred from an extended run, described below, proves the band $K \geq 4$ feasible, so the middle-band *Not Solved* rows in Table 3 are search artifacts, not infeasibility. $K < |\mathcal{I}| = 3$ is infeasible by construction (each active plate needs at least one template via (12)–(13)).

Table 3: K-sweep of the synthetic case study at baseline $\tau_i = 0.005$ under HiGHS, $300\,\mathrm{s}$ per cell. Optimal† = primal feasible but dual gap not closed; Not Solved = no primal found.

K	status	templates used	solve time (s)	total units
3	Infeasible	n/a	44.1	n/a
4	Not Solved	n/a	301.2	n/a
5	Not Solved	n/a	301.4	n/a
6	Not Solved	n/a	301.3	n/a
7	Optimal†	7	301.1	814
8	Optimal†	7	301.2	814
9	Optimal†	6	301.3	814
10	Optimal†	6	301.4	814
11	Optimal†	6	301.4	814

The sweep separates three regimes. At $K = 3$, one template per plate forces the studio equation $37s_M + 12s_T + 6s_C = 58$, which has no non-negative integer solution; the solver proves infeasibility in tens of seconds. At $K \in \{4, 5, 6\}$ the instance sits on a search boundary. HiGHS finds no primal in budget, while Gurobi finds full-program primals at $K = 5, 6$ in under a minute; yet the band is feasible throughout: an extended $1800\,\mathrm{s}$ run

at $K = 11$ returns a four-template, full-program stacking, and since the compactness cap is the only K-dependent constraint, that witness transfers to every $K \geq 4$. The middle band is hard to search, not infeasible; at $T = 5$ the instance also sits at the scaling wall of the "Scaling of V2b" section. From $K \geq 7$ HiGHS finds primals within budget, every feasible cell delivering the full 814-unit program. The zone counts in Table 3 are budget-limited primals, not certified minima, so the four-template stacking is the best known, not a proven minimum (full solver traces in the supplement).

Figure 7 shows the $K = 8$ stacking; because the fixed-count objective is indifferent to vertical arrangement, this run adds the zone-ordering preference of the "Composable extensions" section, which acts at plate granularity (the sequence of templates within a plate is presentational). It reads as a practitioner expects. It uses five templates, one more than the best unordered stacking found, and stays well inside the $K = 8$ budget.

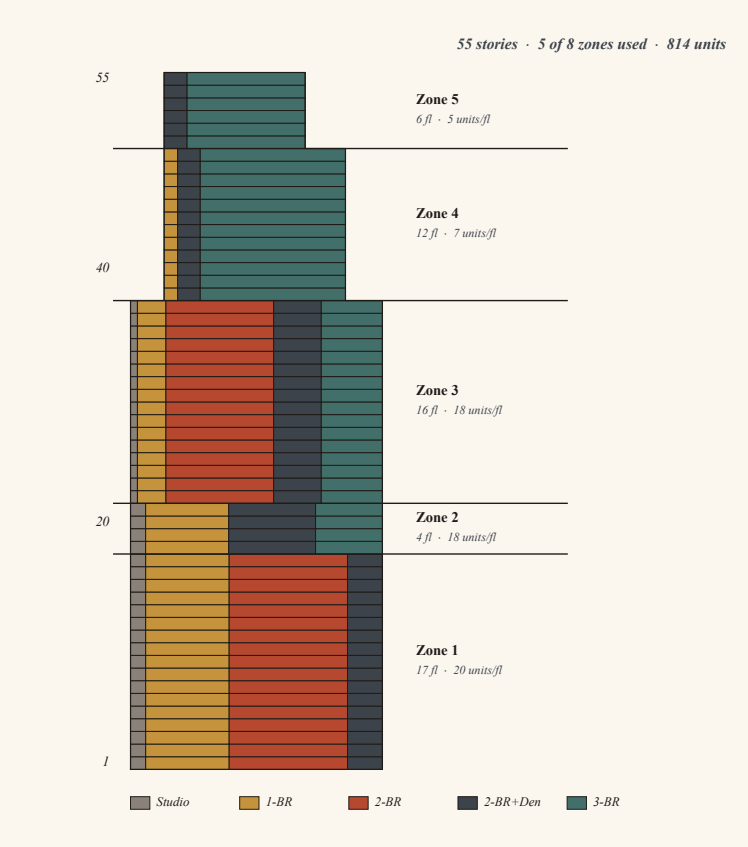

Figure 7: The delivered consultant program (814 units, fixed-count) stacked at $K = 8$ under the zone-ordering preference of the "Composable extensions" section. Mean unit area rises from an affordable base to a three-bed crown while preserving the fixed program.

At the smallest infeasible K, the single-knob diagnostic (the "Infeasibility diagnostic" section) rules out every area-tolerance widening, and the K-sweep locates feasibility from $K = 7$ within budget: area tolerance is not the binding knob, compactness is. That

black-box verdict is exactly what the studio equation above forces: an integer-count blocker no area band can touch. Two independent routes, a closed-form Diophantine argument and an automated single-knob sweep, reach the same verdict, so the diagnostic recovers a structural fact rather than echoing one solver run (full probe rows in the supplement).

7.3 Metaheuristic baseline: what the exact method surfaces that a GA does not

The baseline reimplements the stacking problem as a single-objective genetic algorithm in `pymoo` [46] with integer encoding and a hard-constraint-violation fitness, run on this instance at two settings whose feasibility verdict the K-sweep has just established: at $K = 3$ the program is provably infeasible, at $K = 8$ feasible. The GA cannot tell them apart (Figure 8): 8–13 unit violations at $K = 3$ and 8–11 at $K = 8$, no feasible genome in either case (and at $K = 3$ it also exceeds the template cap, 4–5 active against 3); a $10\times$ compute budget reproduces the same result. A best-effort search carries no certificate; the exact model returns *infeasible* or a certified stacking. The baseline thus validates the headline capability by showing that the dominant alternative cannot supply it (supplemental `heuristic_baseline*.csv`).

7.4 The two-dimensional envelope

Sweeping $K \in \{3, \ldots, 8\}$ against $\tau_M \in \{0, 0.005, 0.01, 0.02\}$ (other plates at 0.005 baseline) under a 240 s budget yields the feasibility envelope of Figure 9.

The three-status return (*optimal*, *infeasible*, *not solved*) maps directly onto the (K, τ_M) plane, letting the architect read off where to spend a relaxation budget: the question "how much main-plate tolerance does a given K require" gets a status-typed answer. A few cells are non-monotone in tolerance ($K = 6$ solves at $\tau_M = 0$ but returns no primal at looser settings within the budget); these are search artifacts of the per-cell budget, and the three-status semantics keeps them visible rather than hiding them.

7.5 Solver-driven program headroom

Fixed-count framing is brittle: a single count off by one mod a plate divisor can Diophantine-block the program (the $K = 3$ studio case). The architect's natural pushback is to relax the program to a per-type minimum and ask how much more fits. Bounded-count mode with a maximize-units objective answers that question directly, sweeping τ_M at $K = 8$ and recording the total unit count returned (Figure 10).

The run uses the same instance and solver; only the count mode and objective change. Because the objective maximizes unit *count*, the cheapest added unit is the

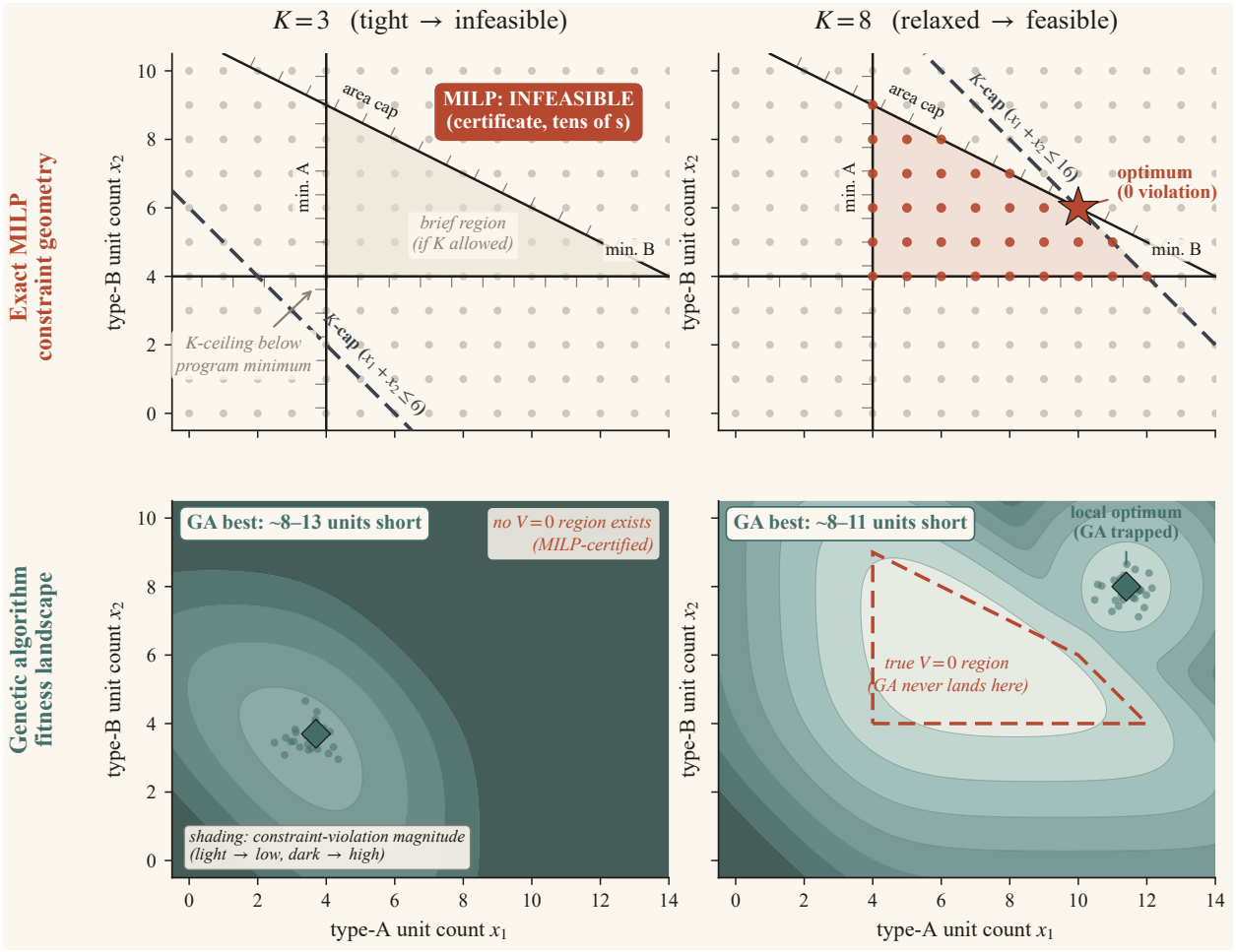

Figure 8: Two readings of one decision space, on a schematic two-type proxy of the count-vector feasible set (not the 55-floor instance to scale); the compactness cap K is the only knob that moves between columns. *Top (exact MILP):* at $K = 3$ the K-ceiling falls below the program's minimum corner, so the feasible region is empty and the solver returns an *infeasible* certificate; at $K = 8$ it is a polygon with the optimum at a zero-violation vertex. *Bottom (genetic algorithm):* the same space as a violation landscape, with a strictly positive floor at $K = 3$ (no feasible point) and a zero-violation region at $K = 8$ the search settles short of. Either way the GA reports a similar-magnitude shortfall (supplemental Table S2), with no signal distinguishing "structurally impossible" from "heuristic miss."

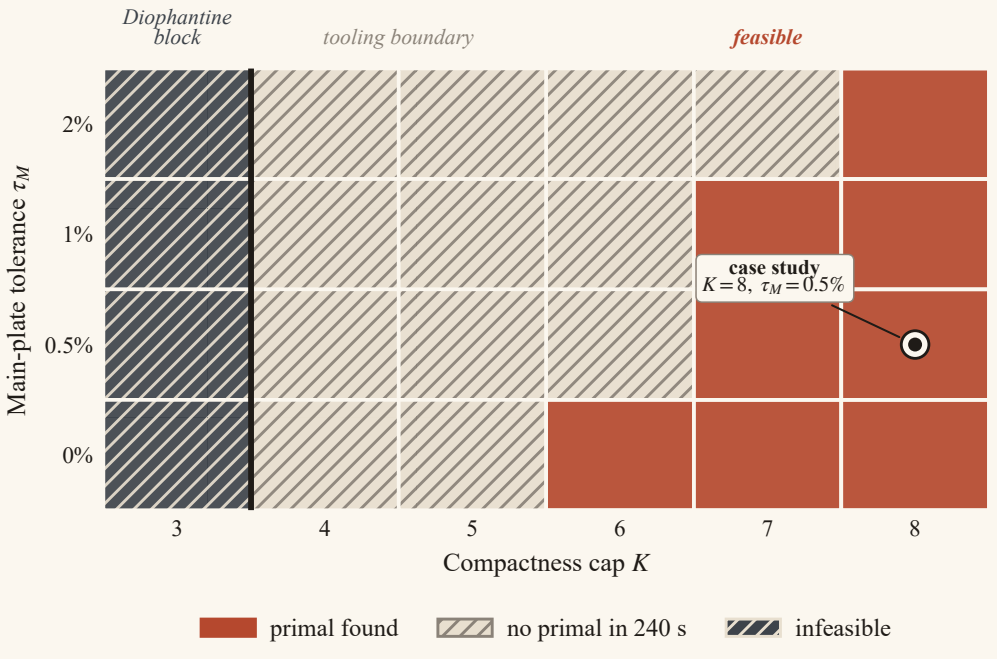

Figure 9: Constraint envelope on the 6×4 grid $K \in \{3, \ldots, 8\} \times \tau_M \in \{0, 0.005, 0.01, 0.02\}$; the operating point ($K = 8$, $\tau_M = 0.5\%$) lies well inside the feasible region.

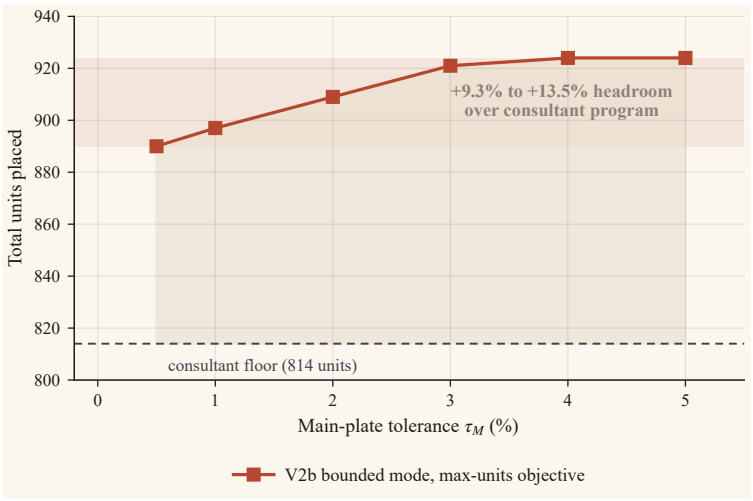

Figure 10: Solver-driven program headroom on the case-study instance at $K = 8$ (bounded-count mode, maximize-units objective): 890 units (+9.3% over the consultant program) at the baseline tolerance, saturating at 924 units (+13.5%).

smallest admissible type, so the curve is a density ceiling under a loose program, not a rentable-area or mix claim.

Across ten synthetic instances (three archetypes with distinct floor divisors at several program-tightness levels, plus the canonical instance), the same bounded-count workflow gives headroom from $+9.3\%$ to $+34.3\%$ (median $+19.8\%$), falling monotonically as the program tightens toward the envelope; the canonical instance is the tightest point, and instances were not selected on outcomes (per-archetype detail in supplemental `case_study_headroom_battery.csv`).

7.6 Architect-in-the-loop workflow

The count-vector model, the diagnostic, and the envelope together support an *architect-in-the-loop* workflow (Figure 2). The architect's role shifts from selecting among generated candidates to negotiating the feasibility envelope against the program, with the three-status verdict marking the boundary explicitly, and the loop re-runs whenever the massing steps differently or the client revises the mix.

8 Real-Building Study: De Piek, Rotterdam

Where the "Synthetic stress test" section pressed the formulation against an adversarial synthetic envelope, this section turns it on a real building under a comfortable, realistic program. The instance is real and metrically grounded; only the unit program is posed.

De Piek (KCAP, Rotterdam, 2025) is a $74\,\mathrm{m}$, 23-storey waterfront tower whose massing *shifts* rather than steps. The floor plate slides horizontally as it rises; the building is a thoroughly documented public example of the heterogeneous-plate regime the "Heterogeneous plate classes" section targets. The plate geometry is traced from the architect's published, scaled section and per-level plans [47], giving dimensionally grounded areas. Three rentable plate classes (net of core and corridor, balconies excluded) carry the program over 19 rentable storeys; a four-floor podium is non-rentable and outside the instance. Table 4 tabulates the full instance.

The published drawings fix the plate geometry but not the unit mix, so the study poses a representative waterfront program informed by local market and housing-policy norms (fixed-count, 157 units): about 29% in the affordable band (studio and one-bed, the *sociale huur* and *middenhuur* range), a two-bed family body, and three-beds with penthouses up top. At nominal areas the program sums to $14{,}772\,\mathrm{m}^2$ against a plate envelope of $14{,}740\,\mathrm{m}^2$, only $+0.2\%$ over: it sits near the envelope with the $\pm 5\%$ per-type band held in reserve.

Figure 11 shows the stacking the solver returns. With three plate classes the compactness cap binds at $K = 3$, one template per architectural plate, and the zone-ordering

Table 4: The De Piek instance: rentable plate classes (left), traced from the architect's published, scaled section, net of the vertical core and corridor; and the posed unit program (right), fixed counts, nominal areas under the same net-area convention, $\sigma_t = 0.05$ for all types. Count-vector enumeration caps per floor: 14 units per type, 18 in total for the stacking run ($15/20$ for the feasibility sweep).

plate class	A_i (m^2, net)	F_i (floors)
lower	940	3
mid	850	8
upper	640	8
Total	n/a	19

unit type	area (m^2)	count	area total (m^2)
studio	50	12	600
1B	68	33	2,244
2B	92	64	5,888
3B	120	40	4,800
PH	155	8	1,240
Total	n/a	157	14,772

preference (the "Composable extensions" section) lands where a practitioner would put it: affordable studios and one-beds at the base, two-bed families in the body, three-beds and penthouses at the crown.

Because the program sits inside the envelope, the feasibility sweep that exposed a boundary on the synthetic instance instead certifies headroom here. Holding the program fixed and tightening the plate (construction) tolerance from the $\pm 2\%$ baseline all the way to fully locked plates leaves the stacking feasible at every step; locking the plates and then tightening the per-type area band from $\pm 5\%$ to $\pm 1\%$ stays feasible throughout (nine cells, all certified on HiGHS). The solver *proves* the program stacks under zero construction tolerance.

A tighter brief on these plates would approach the same boundary, where the diagnostic applies exactly as in the "Synthetic stress test" section; the realistic program posed here stays well clear of it. De Piek is a smaller instance than the synthetic tower, and each feasibility check closes in seconds, well inside interactive range.

9 Discussion

9.1 Implications for design practice

As a development instrument, the feasibility certificate changes what the team agrees to carry forward. A developer can test whether an assumed program is deliverable

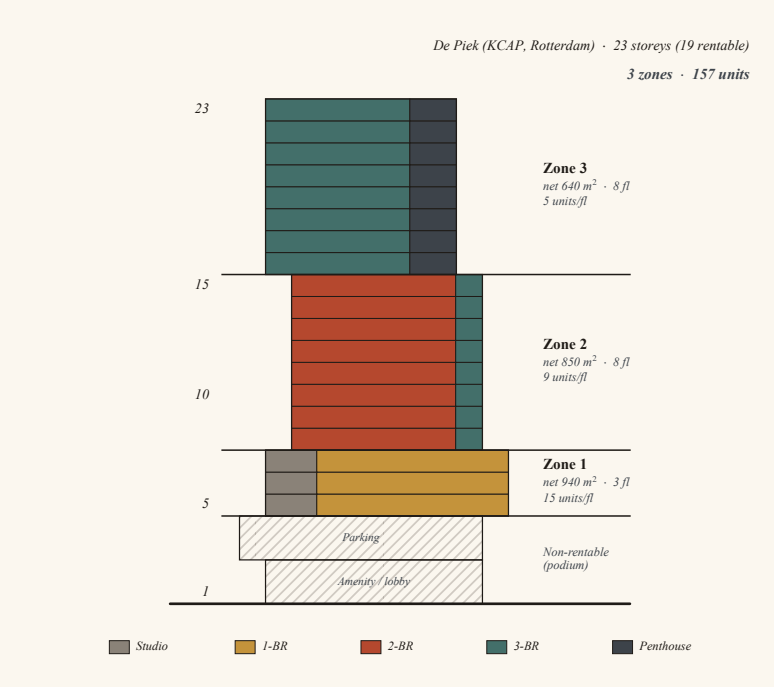

Figure 11: De Piek: the posed 157-unit program stacked at the compact optimum ($K = 3$). The podium is shown for context but is outside the optimization instance.

on the site's massing before committing design fees, and a planning or affordability reviewer can ask whether an inclusionary-housing minimum is achievable on those plates. The single-knob diagnostic names the concession when it is not. Generative tools also gain a principled pre-filter, avoiding searches over programs already proven infeasible.

The template cap has a direct delivery interpretation. Each selected template is a floor-product variant with consequences for unit-plan and structural coordination, facade resolution, service-routing strategy, documentation, procurement, and construction sequencing. The minimum floors-per-template parameter is its minimum production run. Repeated products are what make a consistent wet-wall and riser strategy possible. Feasibility therefore means more than fitting the units: it means fitting them in a product system a developer can repeat.

9.2 Limitations

Five limitations bound the method.

Tight arithmetic cells. The formulation is tractable; some instances remain arithmetically hard. The case-study middle band includes tooling-bound cells ($K = 5, 6$ under HiGHS) and one cell that resists primal search under both solvers ($K = 4$, feasible

by a transferred witness). Both trace to instance-specific plate divisors rather than to a different modeling regime (the "K-sweep and the binding knob" section).

LP-relaxation tightness is empirical. The integrality gap is essentially zero on the bounded-count maximize-units battery (data and regime analysis in the supplemental material). A structural proof remains open, and the proportional-mode gap is a metric artifact of minimize-deviation rather than evidence about the relaxation.

Out of scope: inter-floor coupling and spatial layout. The count-vector abstraction presumes floors within a zone are interchangeable and determines *how many* units of each type a feasible stack admits, not *where* they sit on the plate. Vertical alignment, demising lines, core and structural-grid resolution, orientation and daylighting, and code compliance are downstream test-fit concerns that the count vector constrains and a geometric test-fit resolves. Cost, embodied-carbon, and sequencing layers compose with the skeleton (the "Composable extensions" section), though this paper does not exercise them.

Synthetic batteries plus one real case. The computational batteries use synthetic instances so every parameter can be released, and the real-building study complements them with public architectural documentation. Multi-project validation with architect-endorsed geometry remains future work.

Pool growth at many unit types. Enumeration is bounded by $\prod_t (n_t^{\max} + 1)$, in the 10^2–10^4 range on practical instances but inflatable by loose bands or small unit areas. Most residential programs resolve to a handful of types ($T \leq 4$), where HiGHS closes the V 2b model in seconds to tens of seconds (the "Scaling of V 2b" section); past $T \geq 5$ the pool becomes the binding wall, where column generation or arc-flow reformulation [24, 25] is the established response (the "Future work" section).

9.3 Future work

Vertical alignment introduces inter-template coupling that the count-vector formulation does not natively handle; hybrid template selection plus positional assignment is the substantive direction (the "Composable extensions" section). *Column generation* is the response when the pool exceeds practical size, since each column is already a count vector with linear per-column area variables. Cost, embodied-carbon, BIM integration, and foundation-model baselines belong to a follow-up.

10 Conclusion

This paper formulated unit-mix stacking as a development-feasibility problem. A deliverable program must fit heterogeneous plates and unit-area bands through a limited kit of floor products, each used on a sufficient number of floors. The model supports proportional, bounded, and fixed-count programs and returns an optimal, infeasible, or not-solved verdict. When a program is infeasible, the single-knob diagnostic identifies the smallest actionable relaxation.

Count-vector enumeration makes this instrument computationally practical by dissolving the count $\times$ area and count $\times$ repeated-floor couplings created by variable unit areas and product repetition. The direct McCormick model reaches a $300\,\mathrm{s}$ budget without proving optimality on every $T \geq 3$ benchmark, while the count-vector model closes the benchmark family in seconds to tens of seconds. On the 55-floor stress case, the method identifies template variety rather than plate tolerance as the binding condition and finds 9.3 to 13.5 percent program headroom. On the public De Piek envelope, it certifies a compact stacking with fully locked plates. The result is an exact instrument for negotiating unit programs against the repetition logic through which residential towers are delivered.

Acknowledgments

During the preparation of this work the author used Claude (Anthropic) as an assistive tool to polish wording, formatting, and expression. The author reviewed and edited the content and takes full responsibility for the content of the publication.

Author contributions

Yiliang Shao: conceptualization, methodology, software, validation, formal analysis, investigation, writing (original draft), writing (review and editing), visualization.

Statements and declarations

Ethical considerations

This study did not involve human participants, human data or tissue, or animals. Ethical approval was not required.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Declaration of conflicting interests

The author is employed by Skidmore, Owings & Merrill, an architecture and engineering practice whose projects include residential towers of the kind studied here. The firm provided no funding for this work, holds no commercial interest in the method, and had no role in the study design, the experiments, the interpretation of results, or the decision to publish. No proprietary project data was used. Beyond this employment, the author declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author received no financial support for the research, authorship, and/or publication of this article.

Data availability

The synthetic instance specifications, the solver and experiment scripts, and the result and environment-metadata files underlying every table and figure are openly available at `https://github.com/yiliangs/unit-mix-stacking` and archived at Zenodo [48] (concept DOI `10.5281/zenodo.20583199`; version 1.2.0: `10.5281/zenodo.20652807`). The individual result files cited in the text as supplementary (the per-cell `*.csv` batteries) are contained in that archive. The patent-landscape search summary (the "Patent landscape" section) is provided as a separate supplementary document. The real-building study (the "Real-building study" section) draws only on publicly available architectural documentation cited therein; no proprietary project data was used.

References

[1] United Nations Economic Commission for Europe. Effect of Repetition on Building Operations and Processes on Site. New York: United Nations, Committee on Housing, Building and Planning; 1965. ST/ECE/HOU/14.

[2] Thomas HR, Mathews CT, Ward JG. Learning Curve Models of Construction Productivity. Journal of Construction Engineering and Management. 1986;112(2):245-58.

[3] Bertram N, Fuchs S, Mischke J, Palter R, Strube G, Woetzel J. Modular Construction: From Projects to Products. McKinsey & Company; 2019.

[4] Sidewalk Labs. Delve; 2021. Wayback Machine snapshot: https://web.archive.org/web/20210614110254/https://www.sidewalklabs.com/products/delve.

[5] Autodesk. Autodesk Forma; 2023. https://www.autodesk.com/products/forma.

[6] Finch3D. Finch3D: Generative Floor Plans; 2023. https://www.finch3d.com/.

[7] Fan Z, Liu J, Wang L, Cheng G, Liao M, Liu P, et al. Automated Layout of Modular High-Rise Residential Buildings Based on Genetic Algorithm. Automation in Construction. 2023;152:104943.

[8] Architechtures. Architechtures: AI-Driven Building Design; 2023. https://architechtures.com/.

[9] Aristi Reina F, Tsiliakos M, Kosicki M, Nguyen C, Tsigkari M. From Constraints to Configurations: An Integer Linear Programming Approach for Architectural Space-Planning. In: Ochsendorf J, Mueller C, Fayyad I, Gavriil K, Rabagliati J, editors. Proceedings of the Advances in Architectural Geometry 2025 Symposium. Cambridge, MA, USA; 2025. p. 1-17. Available from: https://discovery.ucl.ac.uk/id/eprint/10224244/.

[10] Zhou X, Luo G, Liao Y, Feng L, Liu J, Qi H, et al. Automated Aggregation of Dwelling Units and Traffic Cores in High-Rise Residential Floor Plans Using Genetic Algorithm and Multi-Agent Cooperative Deep Q-Network. Automation in Construction. 2025;177:106329.

[11] Su P, Lin X, Lu W, Xiong F, Peng Z, Lu Y. Generative Design for Complex Floorplans in High-Rise Residential Buildings: A Monte Carlo Tree Search-Based Self-Organizing Multi-Agent System (MCTS-MAS) Solution. Expert Systems with Applications. 2024;258:125167.

[12] Shi F, Soman RK, Han J, Whyte JK. Addressing Adjacency Constraints in Rectangular Floor Plans Using Monte-Carlo Tree Search. Automation in Construction. 2020;115:103187.

[13] Rahbar M, Mahdavinejad M, Markazi AHD, Bemanian M. Architectural Layout Design Through Deep Learning and Agent-Based Modeling: A Hybrid Approach. Journal of Building Engineering. 2022;47:103822.

[14] Yan S, Liu N. Computational Design of Residential Units' Floor Layout: A Heuristic Algorithm. Journal of Building Engineering. 2024;96:110546.

[15] Weber RE, Mueller C, Reinhart C. Automated Floorplan Generation in Architectural Design: A Review of Methods and Applications. Automation in Construction. 2022;140:104385.

[16] Hu R, Huang Z, Tang Y, van Kaick O, Zhang H, Huang H. Graph2Plan: Learning Floorplan Generation from Layout Graphs. ACM Transactions on Graphics. 2020;39(4):Article 118. Presented at SIGGRAPH 2020.

[17] Aalaei M, Saadi M, Rahbar M, Ekhlassi A. Architectural Layout Generation Using a Graph-Constrained Conditional Generative Adversarial Network (GAN). Automation in Construction. 2023;155:105053.

[18] Bilge EÇ, Yaman H. Multi-Objective Early-Stage Design Optimization of Multi-family Residential Projects. International Journal of Architectural Computing. 2025;23(2):446-63.

[19] Botham CJ. Unit Mix Optimization Tool; 2020. Portfolio page, `https://www.bothamdesign.com/unit-mix-optimization`. Accessed 2026-06-07; page date inferred from site asset metadata.

[20] Singer DJ, Doerry N, Buckley ME. What Is Set-Based Design? Naval Engineers Journal. 2009;121(4):31-43.

[21] Woodbury RF, Burrow AL. Whither Design Space? Artificial Intelligence for Engineering Design, Analysis and Manufacturing. 2006;20(2):63-82.

[22] Gilmore PC, Gomory RE. A Linear Programming Approach to the Cutting-Stock Problem. Operations Research. 1961;9(6):849-59.

[23] Gilmore PC, Gomory RE. A Linear Programming Approach to the Cutting Stock Problem: Part II. Operations Research. 1963;11(6):863-88.

[24] Valério de Carvalho JM. LP Models for Bin Packing and Cutting Stock Problems. European Journal of Operational Research. 2002;141(2):253-73.

[25] Brandão F, Pedroso JP. Bin Packing and Related Problems: General Arc-Flow Formulation with Graph Compression. Computers & Operations Research. 2016;69:56-67.

[26] Goemans MX, Rothvoß T. Polynomiality for Bin Packing with a Constant Number of Item Types. Journal of the ACM. 2020;67(6):Article 38. Preliminary version: arXiv:1307.5108, 2013.

[27] Friesen DK, Langston MA. Variable Sized Bin Packing. SIAM Journal on Computing. 1986;15(1):222-30.

[28] Crainic TG, Perboli G, Rei W, Tadei R. Efficient Lower Bounds and Heuristics for the Variable Cost and Size Bin Packing Problem. Computers & Operations Research. 2011;38(11):1474-82. Predecessor: CIRRELT technical report, 2010.

[29] Brandão F, Pedroso JP. Multiple-Choice Vector Bin Packing: Arc-Flow Formulation with Graph Compression. arXiv; 2013. arXiv:1312.3836.

[30] Patt-Shamir B, Rawitz D. Vector Bin Packing with Multiple-Choice. Discrete Applied Mathematics. 2012;160(10–11):1591-600.

[31] Casazza M, Ceselli A. Exactly Solving Packing Problems with Fragmentation. Computers & Operations Research. 2016;75:202-13.

[32] Correa JR, Epstein L. Bin Packing with Controllable Item Sizes. Information and Computation. 2008;206(8):1003-16.

[33] Hua H, Hovestadt L, Tang P, Li B. Integer Programming for Urban Design. European Journal of Operational Research. 2019;274(3):1125-37.

[34] Klawitter J, Klesen F, Wolff A. Algorithms for Floor Planning with Proximity Requirements. In: Computer-Aided Architectural Design. Design Imperatives: The Future is Now (CAAD Futures 2021). vol. 1465 of Communications in Computer and Information Science. Springer Singapore; 2022. p. 151-71.

[35] Keatruangkamala K, Sinapiromsaran K. Optimizing Architectural Layout Design via Mixed Integer Programming. In: Computer Aided Architectural Design Futures 2005. Springer; 2005. p. 175-84.

[36] Huchette J, Dey SS, Vielma JP. Strong Mixed-Integer Formulations for the Floor Layout Problem. INFOR: Information Systems and Operational Research. 2018;56(4):392-433.

[37] Cubukcuoglu C, Nourian P, Sariyildiz IS, Tasgetiren MF. Optimal Design of New Hospitals: A Computational Workflow for Stacking, Zoning, and Routing. Automation in Construction. 2022;134:104102.

[38] Maciocci G, Bianchini M. System and Method for Automatically Generating an Optimized Building Floor Plate Layout; 2023. Assignee: Lendlease Digital IP Pty Ltd. Filed 2020-12-16; issued 2023-08-15.

[39] McCormick GP. Computability of Global Solutions to Factorable Nonconvex Programs: Part I, Convex Underestimating Problems. Mathematical Programming. 1976;10(1):147-75.

[40] Glover F. Improved Linear Integer Programming Formulations of Nonlinear Integer Problems. Management Science. 1975;22(4):455-60.

[41] Chinneck JW, Dravnieks EW. Locating Minimal Infeasible Constraint Sets in Linear Programs. ORSA Journal on Computing. 1991;3(2):157-68.

[42] Mitchell S, O'Sullivan M, Dunning I. PuLP: A Linear Programming Toolkit for Python. The University of Auckland; 2011. Source: https://github.com/coin-or/pulp.

[43] Huangfu Q, Hall JAJ. Parallelizing the dual revised simplex method. Mathematical Programming Computation. 2018;10(1):119-42.

[44] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual; 2025. Version 13.0. https://www.gurobi.com.

[45] Bestuzheva K, Besançon M, Chen WK, Chmiela A, Donkiewicz T, van Doornmalen J, et al. The SCIP Optimization Suite 8.0. Zuse Institute Berlin; 2021. 21-41. http://nbn-resolving.de/urn:nbn:de:0297-zib-85309.

[46] Blank J, Deb K. pymoo: Multi-Objective Optimization in Python. IEEE Access. 2020;8:89497-509.

[47] KCAP. De Piek Waterfront Residential Tower, Rotterdam; 2026. https://www.archdaily.com/1037217/de-piek-waterfront-residential-tower-kcap. Published 2026-01-10; project completed 2025. Section and per-level plans used for tracing.

[48] Shao Y. unit-mix-stacking: Solver, Synthetic Instances, and Experiment Scripts (v1.2.0); 2026. Zenodo.

A Count-vector enumeration algorithm

Algorithm 1 Count-vector enumeration for a single plate i with area band $[A^{\text{lo}}, A^{\text{hi}}]$ and per-type maxima $n_t^{\max} = \min(N_{\max}, \lfloor A^{\text{hi}}/a_t^{\min} \rfloor)$. Plate-indexed quantities are fixed at the outer call and shown here without the i subscript for clarity.

procedure *EnumerateCV*($A^{\text{lo}}, A^{\text{hi}}, T, n^{\max}, N_{\text{cap}}$)
  $\mathcal{P} \leftarrow \emptyset$
  $n \leftarrow (0, \ldots, 0)$
  Recurse$(1, 0, 0)$
  return $\mathcal{P}$
end procedure
procedure *Recurse*$(j, S^{\min}, S^{\max})$
  if $j = T + 1$ **then**
    if $\sum_t n_t > 0$ **and** $\sum_t n_t \le N_{\text{cap}}$ **and** $S^{\max} \ge A^{\text{lo}}$ **and** $S^{\min} \le A^{\text{hi}}$ **then**
      $\mathcal{P} \leftarrow \mathcal{P} \cup \{(n_1, \ldots, n_T)\}$
    end if
    return
  end if
  for $k = 0, 1, \ldots, n_j^{\max}$ **do**
    $S_{\text{new}}^{\min} \leftarrow S^{\min} + k \cdot a_j^{\min}$
    $S_{\text{new}}^{\max} \leftarrow S^{\max} + k \cdot a_j^{\max}$
    $P \leftarrow \sum_{\ell > j} n_\ell^{\max} a_\ell^{\max}$ ▷ residual upper bound
    if $S_{\text{new}}^{\max} + P < A^{\text{lo}}$ **then**
      continue ▷ underflow prune
    end if
    if $S_{\text{new}}^{\min} > A^{\text{hi}}$ **then**
      break ▷ overflow prune
    end if
    $n_j \leftarrow k$
    Recurse$(j + 1, S_{\text{new}}^{\min}, S_{\text{new}}^{\max})$
  end for
  $n_j \leftarrow 0$
end procedure

B Infeasibility diagnostic algorithm

Algorithm 2 Single-knob infeasibility diagnostic on an instance with parameters $(\boldsymbol{\tau}, \boldsymbol{\sigma}, K, \boldsymbol{N}^\star)$ and time budget Δt per probe.

procedure *Diagnose*($\Theta_0, \Delta\tau, \Delta\sigma, \Delta K, \Delta N$)
  $R \leftarrow \emptyset$ ▷ probe results
  for plate $i \in \mathcal{I}$ **do**
    $\Theta \leftarrow \Theta_0$ with $\tau_i \mathrel{+}= \Delta\tau$
    add (`plate:`i`.area_tol`, $\Delta\tau$, *Solve*(Θ, Δt)) to R
  end for
  for type $t \in \{1, \ldots, T\}$ **do**
    $\Theta \leftarrow \Theta_0$ with $\sigma_t \mathrel{+}= \Delta\sigma$
    add (`unit:`t`.area_tol`, $\Delta\sigma$, *Solve*(Θ, Δt)) to R
  end for
  $\Theta \leftarrow \Theta_0$ with $K \mathrel{+}= \Delta K$
  add (`n_zones`, ΔK, *Solve*(Θ, Δt)) to R
  if mode is `bounded_count` **then**
    for type t **do**
      $\Theta \leftarrow \Theta_0$ with $\underline{N}_t \mathrel{-}= \Delta N$, $\overline{N}_t \mathrel{+}= \Delta N$
      add (`bounded:`t`.window`, $\pm\Delta N$, *Solve*(Θ, Δt)) to R
    end for
  end if
  return R
end procedure

C Synthetic case-study instance

The case-study instance of the "Synthetic stress test" section is fully specified by: the plate classes and unit types with target counts $\boldsymbol{N}^\star$ stated in the "Case-study instance" section and tabulated in Table 2; mode = fixed-count; per-type tolerance $\sigma_t = 0.05$; per-plate tolerance $\tau_i = 0.005$ (baseline; the fully-locked $\tau_M = 0$ stress case is swept in the "Two-dimensional envelope" section); minimum-floors-per-zone $m = 2$; per-floor enumeration caps of 8 units per type and 18 in total; K swept in $\{3, 4, \ldots, 11\}$.

The script `experiments/case_study_ksweep.py` constructs this instance, runs the K-sweep, runs the diagnostic at the smallest infeasible K, and writes results to `experiments/results/case_study_*.csv`. The script `experiments/case_study_envelope.py` writes the data underlying Figure 9. The script `experiments/case_study_headroom.py` reuses the same instance in bounded-count mode with $N_t^\star$ as a per-type minimum and the objective set to maximize total units, writing the data underlying Figure 10. The script `experiments/case_study_k8_stacking.py` solves the same fixed-count instance at $K = 8$ under the zone-ordering objective (the "Composable extensions" section), raising the per-floor caps to 12 units per type and 26 in total so the main plate can host a dense

affordable base and requiring at least two unit types per template so each zone reads as a mix; it writes the data underlying Figure 7. Each figure is plotted from these deposited result files.